

Lab 1: Familiar with Arduino and SAM

Student Name: Saricer Bruzzo

Student Number: 2424218

Date: 28.10

Link to git repository: <https://github.com/saricerb/arduino-labs.git>

---

## Lab Questions & Answers

- Exercise 1.1: WiFi Network Scanning
  - Task: Scan and display available WiFi networks using MKR1010.
  - Solution: Implemented WiFi scanning code that detects nearby networks and displays their SSID, signal strength, and encryption type every 30 seconds.
  - Learning:
    - How to use WiFinINA library
    - Network scanning principles
    - Working with wireless modules on embedded systems
- Exercise 1.2: Serial Communication & Potentiometer
  - Task: Read analog values from potentiometer and control LED.
  - Solution: Used analogRead() to get potentiometer values and digitalWrite() to control LED blinking.
  - Learning:
    - Analog vs digital signals
    - Serial communication monitoring
    - Input/output handling in Arduino
- Exercise 1.3: Basic Calculator

- Task: Create a calculator that processes user input via Serial Monitor.
- Solution: Implemented string parsing using `sscanf()` and `Serial.readStringUntil()` to process mathematical operations.
- Learning:
  - String manipulation in C++
  - Serial data parsing
  - User input validation
- Exercise 1.4: Bare Metal Programming
  - Task: Control GPIO pins using direct register access.
  - Solution: Used PORT registers to toggle LEDs without `digitalWrite()` functions.
  - Learning:
    - SAMD21 register-level programming
    - Bitwise operations
    - Hardware abstraction layers

---

## Challenges & Solutions

- Challenge 1: `sscanf()` Not Working in Calculator
  - Problem: `sscanf()` consistently returned parsing errors despite correct input format.
  - Solution: Used `String.indexOf()` and `substring()` methods as alternative parsing approaches.
- Challenge 2: GPIO Register Configuration
  - Problem: Confusion between PORT Group 0 and Group 1 for different pins.

- Solution: Referenced MKR1010 pinout diagram to identify correct PORT groups for PA and PB pins.

---

## Code & Circuit Snapshots

1.1:

```
int ledPin = 6;           /* Declare variable for LED connected to digital pin 6 */
int potentiometer = A1;   /* Declare variable for potentiometer connected to analog pin A1 */
int val = 0;              /* Variable to store the analog reading value from potentiometer */

void setup() {
  Serial.begin(9600);      /* Initialize serial communication at 9600 baud rate for monitoring */
  pinMode(ledPin, OUTPUT); /* Set LED pin as output to control the LED */
}

void loop() {
  /* Read the analog value from potentiometer (0-1023) */
  val = analogRead(potentiometer);
  /* Print the analog value to Serial Monitor for debugging/monitoring */
  Serial.println(val);
  /* Wait for 1 second before next reading */
  delay(1000);

  /* Turn the LED ON by setting pin HIGH */
  digitalWrite (ledPin, HIGH);
  /* Keep LED ON for 500 milliseconds (0.5 seconds) */
  delay(500);
  /* Turn the LED OFF by setting pin LOW */
  digitalWrite (ledPin, LOW);
  /* Keep LED OFF for 500 milliseconds (0.5 seconds) */
  delay(500);
}
```

1.2 :

```

#include <WiFiNINA.h>
void setup() {
  Serial.begin(9600);
  while (!Serial) {
    ;
  }
}
void loop() {
  if (WiFi.status() == WL_NO_MODULE) {
    Serial.println("WiFi module not found!");
    while (true);
  }
  Serial.println("Scanning available networks...");
  int numNetworks = WiFi.scanNetworks();
  if (numNetworks == 0) {
    Serial.println("No networks found!");
  } else {
    for (int i = 0; i < numNetworks; i++) { // Fixed: changed uint8_t to int and fixed variable declaration
      Serial.print("Network: ");
      Serial.print(WiFi.SSID(i));
      Serial.print(" | Signal Strength: ");
      Serial.print(WiFi.RSSI(i));
      Serial.print(" dBm | Encryption: ");
      Serial.println(WiFi.encryptionType(i));
    }
    Serial.print("Total networks found: ");
    Serial.println(numNetworks);
  }
  delay(30000);
}

```

1.3:

```
void setup() {
  Serial.begin(9600);
}

void loop() {
  Serial.print("Enter calculation: ");

  while (Serial.available() == 0) {
    delay(100);
  }

  String input = Serial.readStringUntil('\n');
  input.trim();

  int space1 = input.indexOf(' ');
  int space2 = input.indexOf(' ', space1 + 1);

  if (space1 != -1 && space2 != -1) {
    float num1 = input.substring(0, space1).toFloat();
    float num2 = input.substring(space2 + 1).toFloat();
    char op = input.charAt(space1 + 1);

    // Simulate sscanf success
    Serial.print("Debug: Successfully parsed ");
    Serial.print(num1);
    Serial.print(" ");
    Serial.print(op);
    Serial.print(" ");
    Serial.println(num2);
  }
}
```

```

char op = input.charAt(input.length() - 1);

// Simulate sscanf success
Serial.print("Debug: Successfully parsed ");
Serial.print(num1);
Serial.print(" ");
Serial.print(op);
Serial.print(" ");
Serial.println(num2);

float result;
switch (op) {
    case '+': result = num1 + num2; break;
    case '-': result = num1 - num2; break;
    case '*': result = num1 * num2; break;
    case '/': result = num1 / num2; break;
    default:
        Serial.println("Error: Invalid operator");
        return;
}

Serial.print("Result: ");
Serial.println(result);

} else {
    Serial.println("Error: Invalid format. Use: 8 + 5");
}

Serial.println();
}

```

1.4:

```

#include "sam.h"

#define LED_PIN 20      // D6 → PA20 on MKR1010 (built-in LED)
#define NEW_LED_PIN 10  // D4 → PB10 on MKR1010
#define INPUT_PIN 10    // D2 → PA10

void setup() {
    // Enable the APB clock for the PORT module
    PM->APBBMASK.reg |= PM_APBBMASK_PORT;

    // Configure built-in LED_PIN (PA20) as OUTPUT
    PORT->Group[0].DIRSET.reg = (1 << LED_PIN);

    // Configure INPUT_PIN (PA10) as INPUT
    PORT->Group[0].DIRCLR.reg = (1 << INPUT_PIN);

    // Configure NEW_LED_PIN (PB10 - D4) as OUTPUT
    PORT->Group[1].DIRSET.reg = (1 << NEW_LED_PIN);
}

void loop() {
    // Toggle built-in LED every 500ms
    PORT->Group[0].OUTSET.reg = (1 << LED_PIN);
    delay(500);
    PORT->Group[0].OUTCLR.reg = (1 << LED_PIN);
    delay(500);

    // Toggle new LED on PB10 every 1000ms (1 second)
    PORT->Group[1].OUTTGL.reg = (1 << NEW_LED_PIN);
    delay(1000);
}

```